

CODE SPORT



Sandya Mannarswamy

Welcome to CodeSport! In this month's column, we continue our discussion on memory access errors.

In last month's column, we had discussed a set of questions that could prove helpful to our student readers, as they prepare for interviews. While we have had a number of responses to the questions, I did not get answers to all, so I will keep the questions open for another month, and discuss the solutions in the next issue of *LFY*.

In my March column, I had discussed various programming errors that can lead to coding bugs. These included memory access/usage errors such as memory leaks and Uninitialised Memory Reads (UMR). A few readers had requested me to cover the tools to detect memory access errors. Hence, in this month's column, I will discuss various memory related programming errors and the tools used to detect them.

What are the different types of memory errors?

An application's data can be stored in global variables, function-local variables on the stack, or in dynamically allocated heap memory. Dynamically allocated heap memory allows the programmer to allocate the memory at runtime, depending on the actual application requirements—thereby providing considerable flexibility to the programmer. In programming languages without explicit garbage collection support like C/C++, the programmer makes the decisions about which point in the application to allocate memory on heap, how much memory to allocate, and when to free the memory once the intended use of the allocated memory is over.

The price for the programming flexibility provided by dynamically allocated heap memory is the increased possibility of the programmer introducing coding errors when handling heap memory. This is unlike statically allocated global data, which is not freed during the application

lifetime, or function-local memory allocated on the stack, which is automatically freed on returning from the function. Since programmers decide the lifetime of the dynamically allocated heap memory, they can make a number of errors, like freeing the memory too early (before it is fully used); forgetting to free the memory at all, which results in a memory leak; or using uninitialised heap memory in the application. The two major classes of memory errors that cause the most headaches for programmers are invalid memory access and memory leaks. The various classes of invalid memory access are:

- Out-of-bound memory access, also known as buffer over-runs and under-runs.
- Read/write to freed memory, known as 'use after free' errors.
- Attempting to free memory that is already freed; known as a 'double free' error.
- Attempting to free memory that is not dynamically allocated; known as 'bad free'.
- Uninitialised memory reads.
- Mismatch in allocation/de-allocation calls to the memory allocator, such as trying to free a memory block allocated using 'new', or trying to 'delete' memory allocated through 'malloc'.
- Overlapping of source and destination pointers in *memcpy/memset/strcpy* and other related functions.

First, let us look at a few examples of invalid memory accesses, and see if you can find and identify the issue in each of these cases.

```
foo() {  
  
    char *buf;  
  
    buf = malloc(100);
```